

四足组培训任务提交与验收规范

1. 目的

本规范用于统一四足组培训期间各阶段任务的提交、检查与验收方式。

培训任务会随着课程逐步推进，并持续积累为后续机器人开发所需的完整工程基础。因此，任务提交除了用于确认完成情况，也用于培养基本的工程习惯，包括：

- 使用 Git 管理代码与修改记录；
- 使用 GitHub 保存和同步个人培训工程；
- 编写基本的项目说明文档；
- 保持清晰的目录和代码结构；
- 能够独立运行、解释和修改自己完成的程序。

整体采用 **GitHub 提交 + 当面验收** 的方式。GitHub 主要用于任务提交、过程留档和代码检查，当面验收主要用于确认任务能够实际运行，并确认本人理解所完成的内容。

2. 个人培训仓库

每位成员建立一个个人培训仓库，整个培训阶段持续使用，不需要每次任务重新创建仓库。

仓库名称建议为：

quadruped-training-姓名或GitHub用户名

推荐目录结构：

```
quadruped-training-xxx/  
├─ README.md  
├─ 01_xxx/  
├─ 02_xxx/  
├─ 03_xxx/  
├─ 04_xxx/  
└─ ...
```

每次培训任务放在独立目录中，目录序号与培训顺序保持一致。具体目录名称可以根据任务内容调整。

培训期间的小任务统一放在该仓库中；后续若进入相对独立的大作业、队内赛或正式项目，可再单独建立工程仓库。

3. Git 与 GitHub 基本要求

从需要正式提交的培训任务开始，统一通过 Git 管理并上传至 GitHub。

至少需要掌握以下基本操作：

```
git clone  
git status  
git add  
git commit  
git pull  
git push
```

要求能够自行完成代码修改、提交和同步，不要求在培训初期掌握复杂的分支管理、rebase 等操作。

3.1 提交记录

应尽量在开发过程中分阶段提交，例如：

完成基础工程搭建
实现模型加载
增加控制功能
调整程序结构
补充 README

不建议在任务全部完成后只进行一次提交。

Commit 信息不要求严格遵循正式项目规范，但应能够大致说明本次修改内容，避免长期使用：

```
update  
test  
123  
final
```

等无法说明修改内容的提交信息。

3.2 GitHub 仓库可访问性

仓库可以设置为公开仓库。

如果使用 Private 仓库，应确保负责培训和验收的成员具有访问权限。

4. 每次任务的提交内容

除任务本身另有要求外，一次完整提交通常应包含以下内容：

4.1 源代码与配置

提交完成任务所需的代码、配置文件、脚本和必要的小型资源文件。

目录结构应尽量清晰，避免所有文件无分类堆放在同一目录中。

4.2 README

每个正式任务目录原则上应包含一个 `README.md`，至少说明：

1. 本次任务完成了什么；
2. 使用的主要环境和依赖；
3. 程序如何编译或运行；
4. 主要文件或目录的作用；
5. 必要的补充说明。

README 不需要写成完整技术报告，以能够让其他人按照说明理解并运行项目为标准。

4.3 运行结果

根据任务类型，可以提交适量的运行结果，例如：

- 程序运行截图；
- 仿真截图；
- 终端输出；
- 简短日志；
- 必要的视频链接。

运行结果主要用于快速确认任务完成情况，不要求大量堆积截图。

4.4 不应提交的内容

原则上不要上传：

- 编译生成的大量中间文件；
- IDE 缓存；
- Python 缓存；
- 无关数据集；
- 大型二进制文件；
- 系统文件；

- 密钥、Token、账号密码等敏感信息。

应逐步学习并使用 `.gitignore` 排除不需要进入版本管理的文件。

对于体积较大的模型、数据或第三方资源，可以在 README 中说明获取方式，不必全部上传到仓库。

5. 任务提交方式

每次需要验收的任务，在规定时间内完成以下内容：

1. 完成任务；
2. 将最新版本提交到个人培训仓库；
3. 确保 GitHub 上能够看到最新提交；
4. 按当次培训要求登记仓库链接、目录位置或最新 commit；
5. 准备现场运行环境。

一般情况下，不需要另外打包压缩文件提交。

任务以 GitHub 上规定时间前的提交状态作为主要留档版本。若之后进行了修改，应继续正常 commit，不要通过删除历史记录的方式覆盖原有过程。

6. 当面验收

对于需要正式验收的任务，原则上采用当面验收。

验收主要检查以下内容：

- 任务要求的主要功能是否完成；
- 程序能否在本人环境中正常运行；
- 提交内容与现场运行内容是否一致；
- 本人是否理解代码的基本结构和关键部分；
- 能否解释任务中涉及的基础概念；
- 能否完成简单的现场修改或排查。

验收问题主要围绕本人完成的任务和代码，不以考察冷门知识或死记概念为目的。

现场验收时间根据任务规模确定，小型任务应尽量控制在较短时间内。

7. 验收结果

验收结果原则上分为三种：

7.1 通过

任务主要要求已经完成，程序能够正常运行，对自己完成的内容有基本理解。

允许存在少量不影响主要功能的问题。

7.2 修改后通过

主要任务已经完成，但存在较明显的小问题，例如：

- README 不完整；
- 仓库结构较混乱；
- 部分代码需要整理；
- 个别功能存在小错误；
- 对部分内容理解不够清楚。

按照要求修改后即可完成本阶段任务，一般不需要重新进行完整验收。

7.3 补验

出现以下情况时，需要完成修改后重新验收：

- 主要任务未完成；
- 程序无法正常运行；
- 提交内容明显不完整；
- 无法解释自己提交代码的基本逻辑；

- 大量直接复制代码且本人不了解其内容；
- 未按要求完成基本的 GitHub 提交。

补验的目的主要是保证后续培训具备必要基础，不作为单纯的惩罚措施。

8. 关于 AI 的使用

培训过程中允许使用 AI 辅助学习，包括：

- 解释概念；
- 分析报错；
- 帮助查找问题；
- 提供代码修改建议；
- 解释不理解的代码；
- 帮助整理思路和文档。

学习阶段不建议直接让 AI 一次性生成完整任务后直接提交。

无论代码来源于自己编写、官方示例、开源项目还是 AI，最终都应能够理解自己提交的主要内容。验收时会以实际理解程度为准，不要求能够逐行背诵代码。

使用外部代码或资料时，应保留基本的来源说明。

9. 代码与工程要求

培训初期不要求达到正式项目的工程质量，但应逐步形成以下习惯：

- 文件和目录命名清晰；
- 不将所有功能长期写在单个超大文件中；
- 不随意使用大量绝对路径；
- 不将环境相关参数全部硬编码；
- 保留必要的注释；
- 删除明显无用的测试代码；

- 保证 README 中的运行方式与实际一致；
- 尽量保证他人能够根据仓库内容理解项目结构。

随着培训推进，对代码结构和工程规范的要求可以逐步提高。

10. 缺交、延期与补交

如果因课程、比赛、设备或其他合理原因无法按时完成任务，应尽量提前说明情况。

延期任务仍需要按照正常方式提交至个人仓库，并在完成后进行补验。

不建议通过集中复制其他成员代码的方式赶进度。如果某一阶段基础没有掌握，应优先解决该阶段问题，否则容易影响后续任务。

11. 培训仓库的长期使用

个人培训仓库应尽量完整保留整个培训过程。

随着培训推进，仓库将逐步记录从基础开发环境、编程与仿真，到电机控制、通信、ROS2、强化学习部署与训练等内容。

培训结束后，该仓库既可以作为个人学习记录，也可以用于后续复习和排查问题。

正式进入队内项目后，再根据实际开发需要使用独立项目仓库和更严格的 Git 协作规范。

12. 总体原则

培训任务的验收重点包括三点：

做出来、能运行、讲得清楚。

GitHub 用来记录完成过程和工程内容，当面验收用来确认实际能力。培训阶段不追求形式复杂的评分体系，也不要求为了验收额外制作大量材料。

规范的目的，是让每个人在完成任务的同时逐步建立能够用于后续机器人开发的基本工程习惯。